Offset specification

[BX | BP] + [SI | DI] + [const] **ON 16 BITS**

based addressing — indexed addressing — direct addressing

At assembly time we know the offset.
The second part is only known on linking time (by the OS)

mov ax, [eax + ebp * 4 - 7] => valid (32 bits specification)
mov eax, [bx + si + 6] (16 bits specification)

NO MIXTURE OF THOSE. ONLY 1 OF THEM.

Machine Language = the lang. where the processor directly reads. Represented as bit strings with predefined semantics.

Assembly Language - basic instructions that correspond to valid machine code. This is a symbolic lang.

* labels = user defined names pointing to memory areas
* instructions = mnemonics that suggest actions. (mov, add)
* directives = indications (not directed to CPU) directed to the compiler & assembler
* location counter = "$" - integer number managed by the assembler => specifies the current offset.

JMP $ ⇒ infinite loop.

Specifically to NASM is $$ - starting of the current section.
How far you are = ($ - $$)

```
section .data
    db 'hello'
    data_segment_size equ $ - $$
```

This would be 5 (5 bytes are defined)

```
a   db  17, -2, ...
    db  10, 8, ...
b   dw ...

mov eax, b - a ; pointer arithmetic
```

OR

```
a   db  17, -2, ...      symbolic
    db  10, 8, ...   ->  Constant (=> no memory allocated)
lg_a equ $ - a
```

or

```
lg_a dw lg_a - a
```

lg_a is already defined.

lg_a: variable

The assembler/compiler has 2 main tasks.

1. Check the sintactic correctness & validity of our source code.

!!! 2. Generating the corresponding bytes for the instructions & directives met in the source code.

All EQU are placed into the table of constants (special memory address)

⇒ ~~mov eax, [lg_a]~~ ✗

best ~~mov eax, lg_a~~ ✓

repz cmpsd ; label + mnemonic + comment.

↑ prefix

here: jmp here ; label + mnemonic + operand + comment

a dw 1984, 14 ; label + mnemonic + 2 operands.

segment code

start:

jmp real_start:

a db ...

b dw ....

c dd ...

real_start:

mov eax, [ebx+2] ✓

mov eax, ebx+2 ✗ error

mov eax, [V+2] ✓

mov eax, [V]+2 ✗ error.

loading time ⇒ loads into memory.

only after it's fixed into memory, the OS can know the segment part.

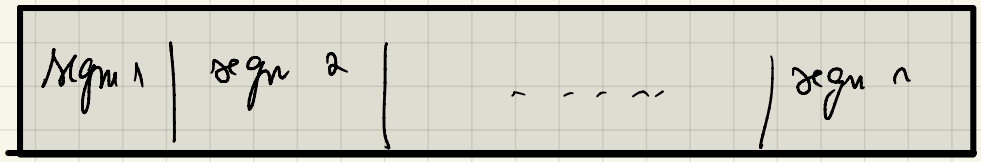
?? : offset 1
?? : offset 2
) until loading time

address relocation performed by the loader

linker checks for the validity of variables.

ex: variable "a" is unique through modules.

1. compiling
2. linking
3. loading
4. relocation.



↑

.exe start.

the linker knows the offset.

CS, DS, ES, SS.

— every offset is part of one of the 4 types of active segments.

JMP FAR CS:...

JMP FAR DS:... or JMP FAR [label2]

ES can only be used in explicit specifications like ES:[Var]

mov EAX,[V] mov eax, DWORD PTR DS:[40500]

CS => for code labels targets (jmp, call, ret, etc.)

SS => in SIB addressing using EBP or ESP as base.

DS => for rest.

mov eax, [ebx] ; DS:[ebx]

mov eax, [ebp] SS

mov eax, [ebp*2] DWORD PTR

[ebp*3] SS:[ebp+ebp]

[ebp*4] -11- [ebp+ebp*2]

DS:[ebp*4]

mov eax, [ebx + esp] mov eax, DWORD PTR [SS: esp + ebx]

mov eax, [esp + ebx] -//-

[ebx + esp*2] → syntax error.

[ebx + ebp*2] - DS

BASE [ebx + ebp] - DS

[ebp + ebx] - SS.

[ebx + 2 * ebp] - SS

[ebx + 1 + esp] - SS

[ebp + 1 + ebx] - DS

Considers this as a scale.

[ebx * 1 + ebp*1] - SS

[ebp + 1 + ebx*1] - DS

jmp et 1; jmp ~~start~~ 00: 1024